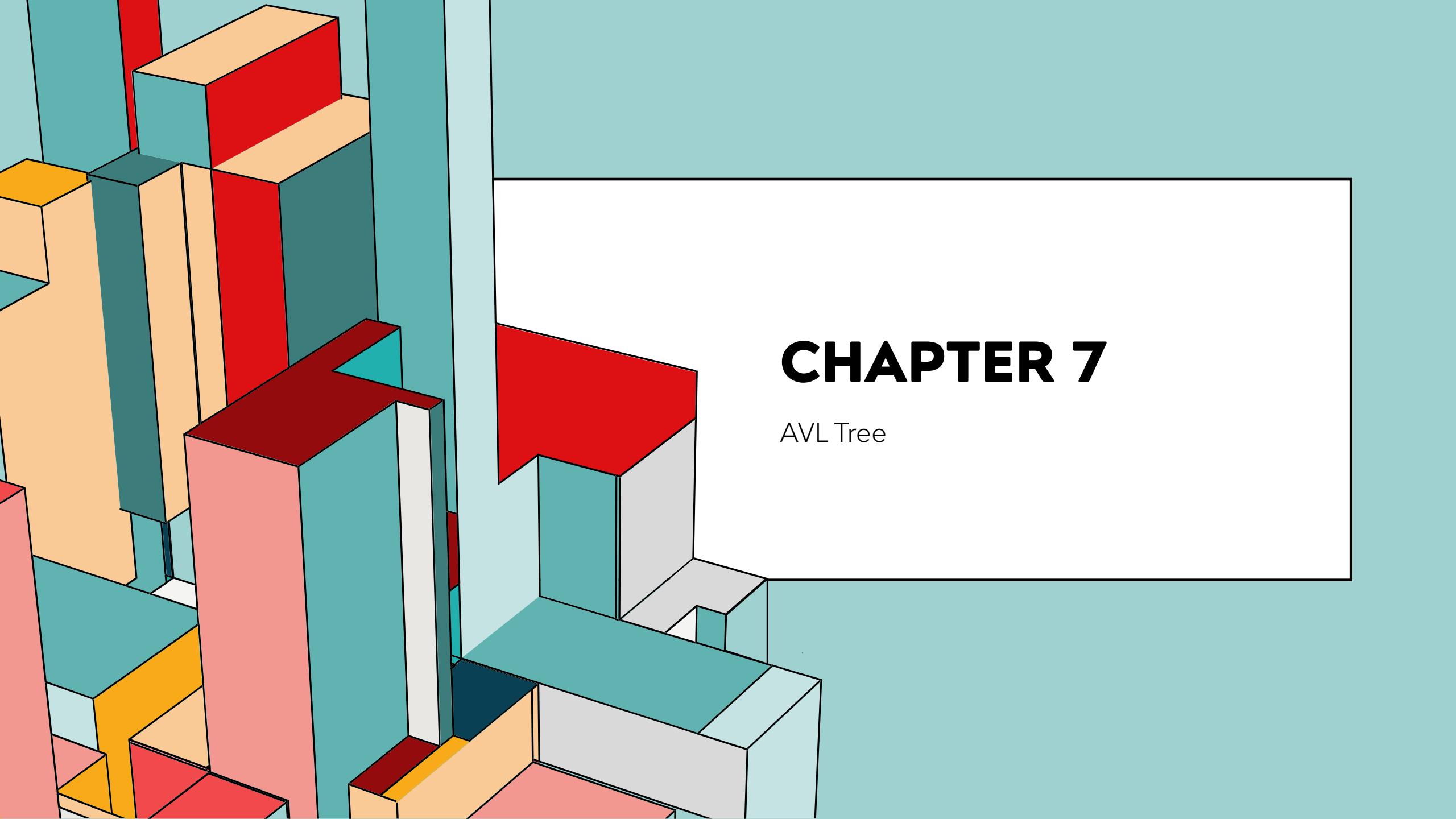


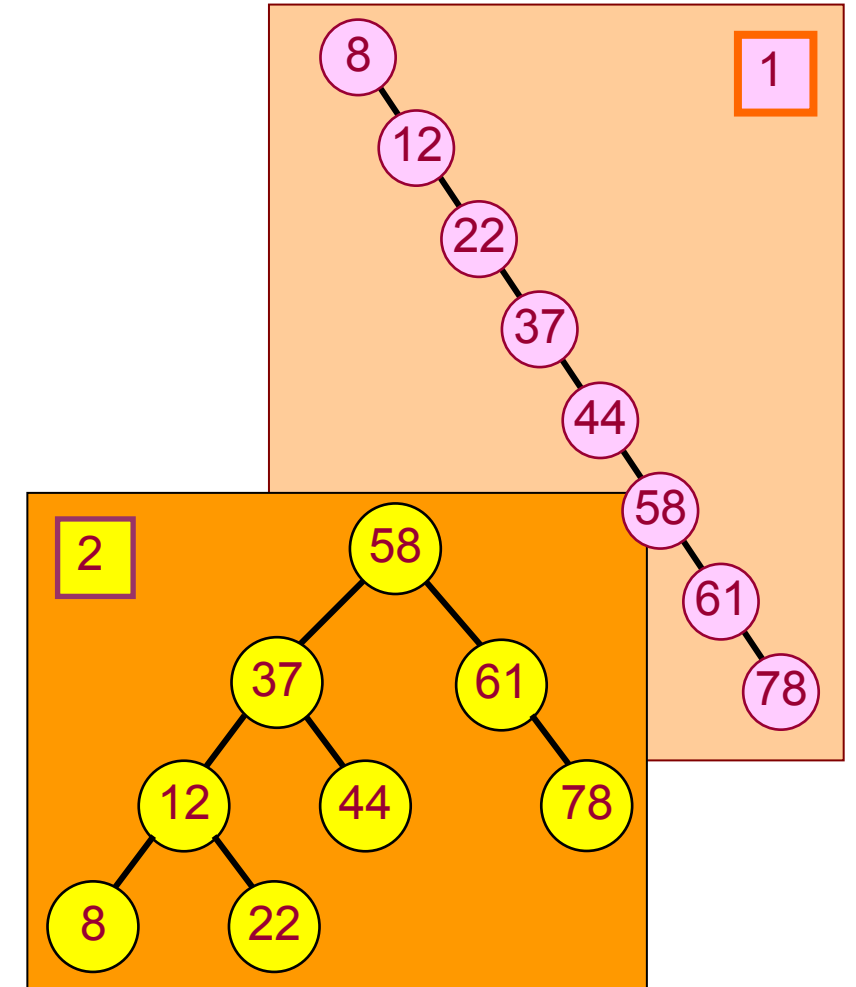


CHAPTER 7

AVL Tree

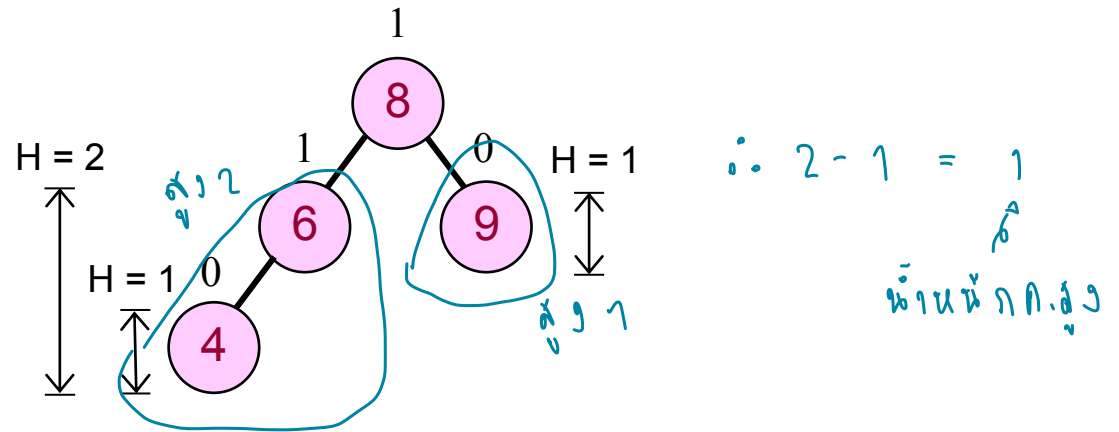
BINARY SEARCH TREE

- ปัญหา : BST อาจจะไม่สมดุล (เบ้ซ้าย-ขวา) ขึ้นกับลำดับของการแทรกข้อมูล
- ความไม่สมดุลนี้ ทำให้ประสิทธิภาพการค้นหาข้อมูลแย่ลง เช่น ถ้าต้องการค้นหาข้อมูล 78
 - ต้นไม้แบบแรก จะต้องเปรียบเทียบโหนดถึง 8 ครั้ง จึงพบโหนดที่มีค่า 78
 - ต้นไม้แบบที่สอง จะต้องเปรียบเทียบโหนดเพียง 3 ครั้ง จึงพบโหนดที่มีค่า 78
 - แบบแรกค้นหาได้ช้ากว่าแบบที่สอง



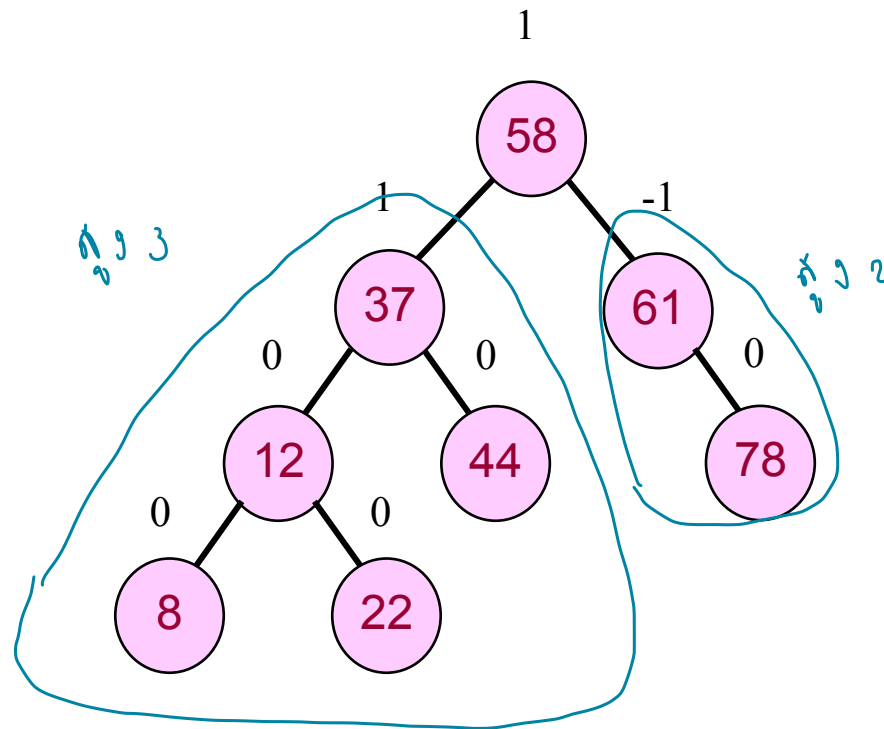
AVL TREE

- AVL Tree : เป็น BST ที่มีความสมดุล (Balance)
- ต้นไม้สมดุล : ความสูงของต้นไม้ย่อยข้างซ้ายและขวาต่างกันไม่เกิน 1 ระดับในทุกๆ โหนด ซึ่งสามารถกำหนดน้ำหนักความสูงเป็น -1, 0, 1
- น้ำหนักความสูง = ความสูงของ Left Subtree - ความสูงของ Right Subtree



EXAMPLE : AVL TREE

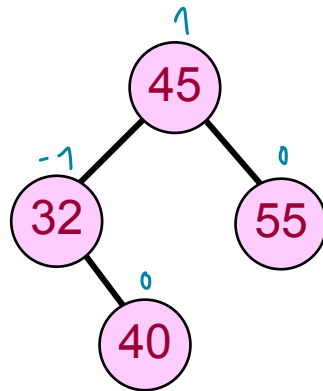
- ทุกโหนดมีค่าน้ำหนักความสูง -1 หรือ 0 หรือ 1



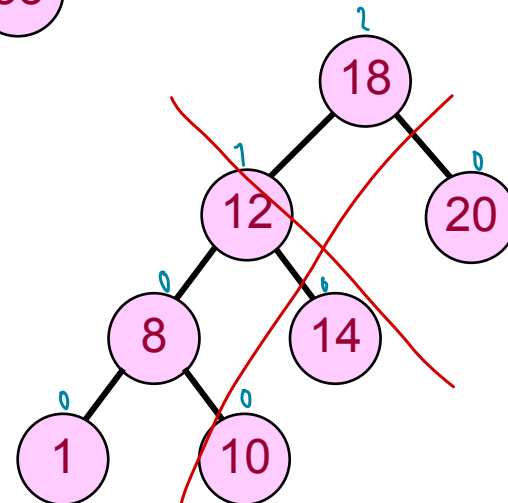
∴ น้ำหนักความสูง $3 - 2 = 1$

EXAMPLE : AVL TREE

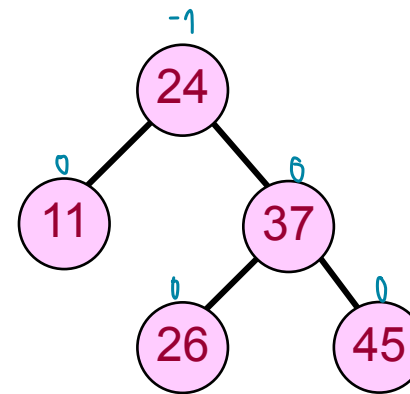
- ให้เขียนค่าน้ำหนักความสูงในทุกโหนด



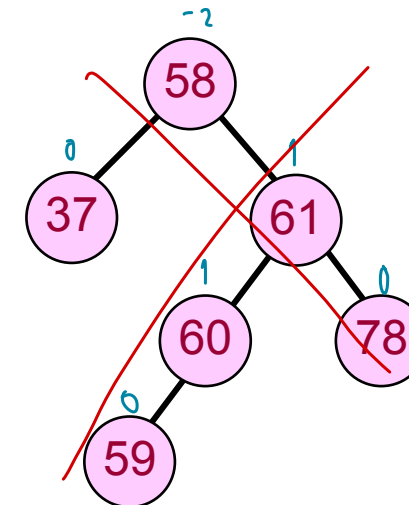
(a)



(c)



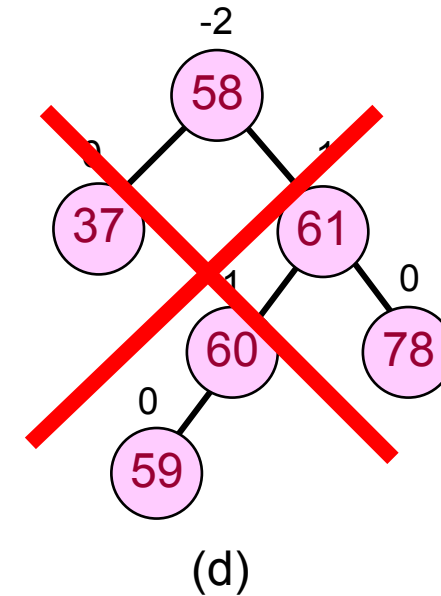
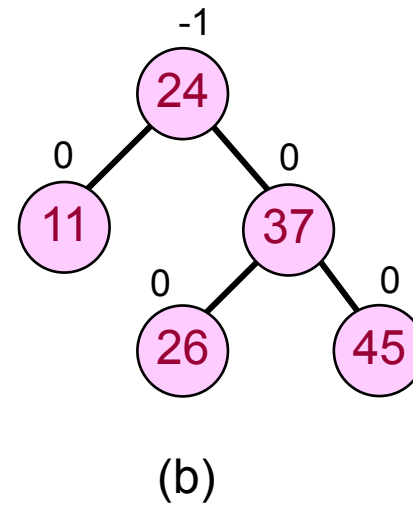
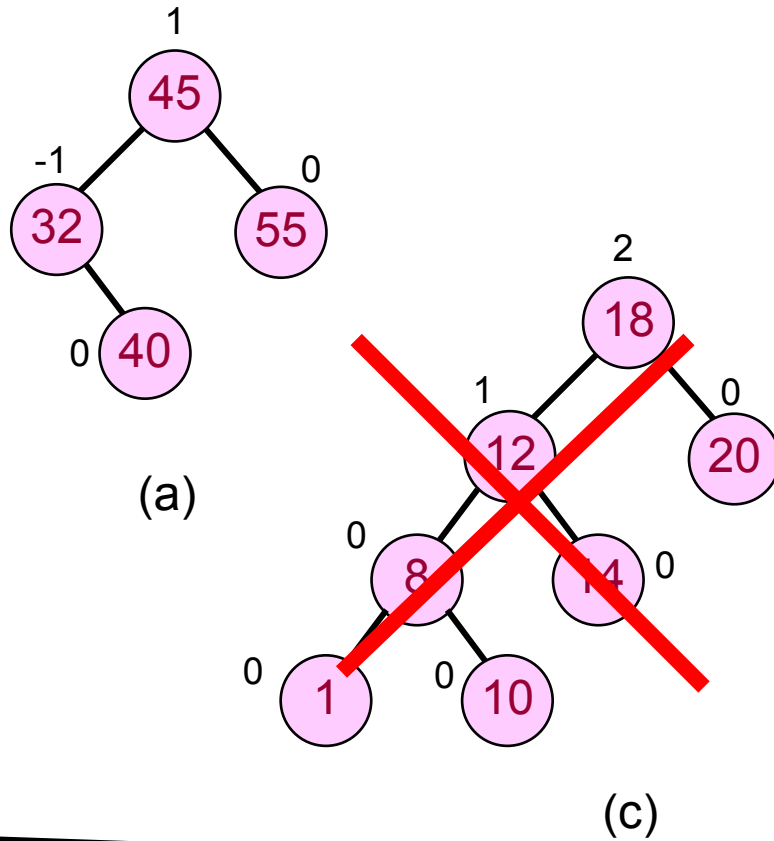
(b)



(d)

EXAMPLE : AVL TREE

- ให้เขียนค่าน้ำหนักความสูงในทุกโหนด



BALANCING TREES

- เมื่อแทรกหรือลบโหนดที่ BST อาจเป็นทำให้ BST ไม่สมดุลได้
- เมื่อ BST ไม่สมดุลแล้ว เราสามารถปรับสมดุลของ BST ได้โดยการหมุนโหนดที่มีค่าน้ำหนักไม่เหมาะสม แบ่งเป็น 4 แบบ

หมุน 1 ครั้ง

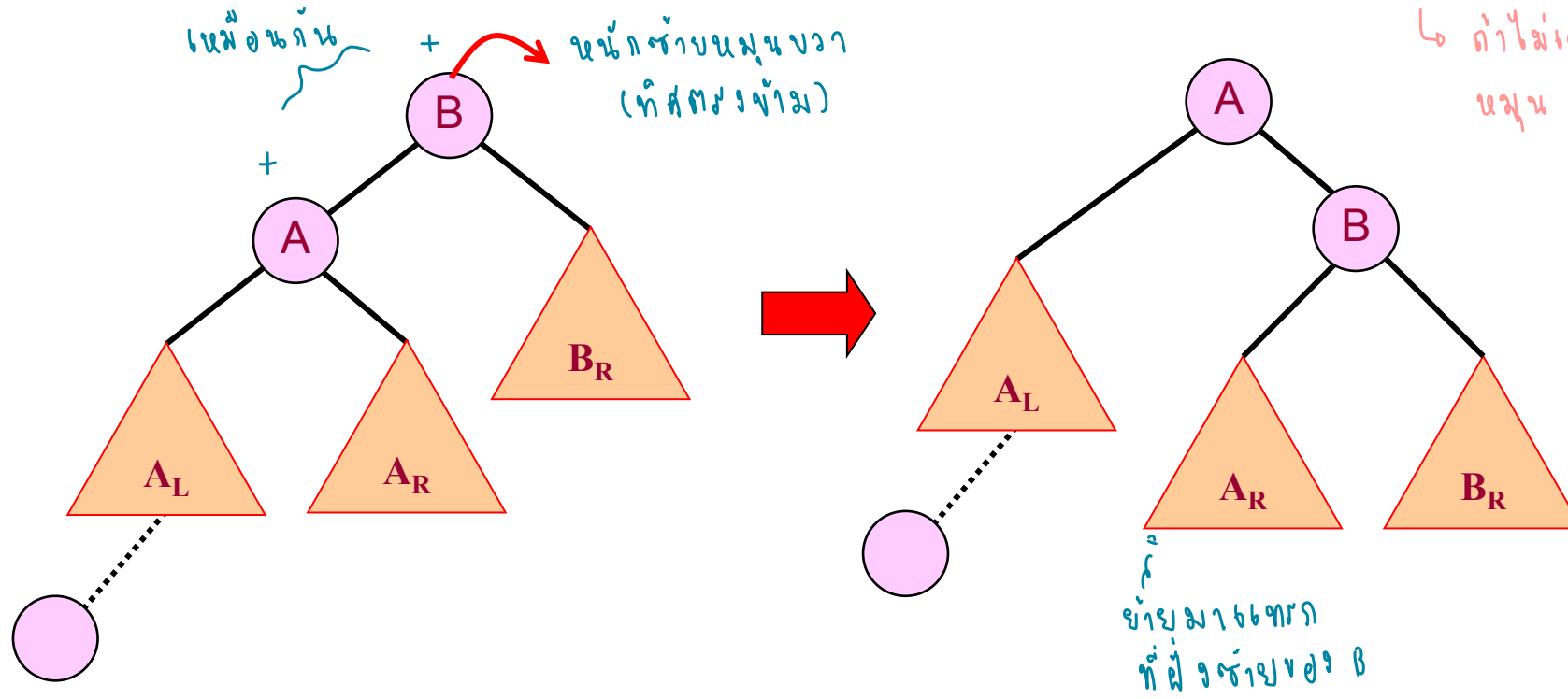
- หมุนขวา 1 ครั้ง
- หมุนซ้าย 1 ครั้ง

หมุน 2 ครั้ง

- หมุนซ้ายที่โหนดลูก แล้วหมุนขวาโหนดที่มีค่าน้ำหนักไม่เหมาะสม
- หมุนขวาที่โหนดลูก แล้วหมุนซ้ายโหนดที่มีค่าน้ำหนักไม่เหมาะสม

CASE 1 : LEFT TO LEFT

- หมุนขวา 1 ครั้ง (หมุนโหนด B)
 - โหนด B และ A เบ้ซ้าย (ค่าน้ำหนักเป็นบวกทั้งคู่)



- ดู Node ที่ไม่ใช่ (0, 1, -1)
- ดูลูกฝั่งที่เอียงของ Node นั้น
ว่าไปทิศทางเดียวกันไหม

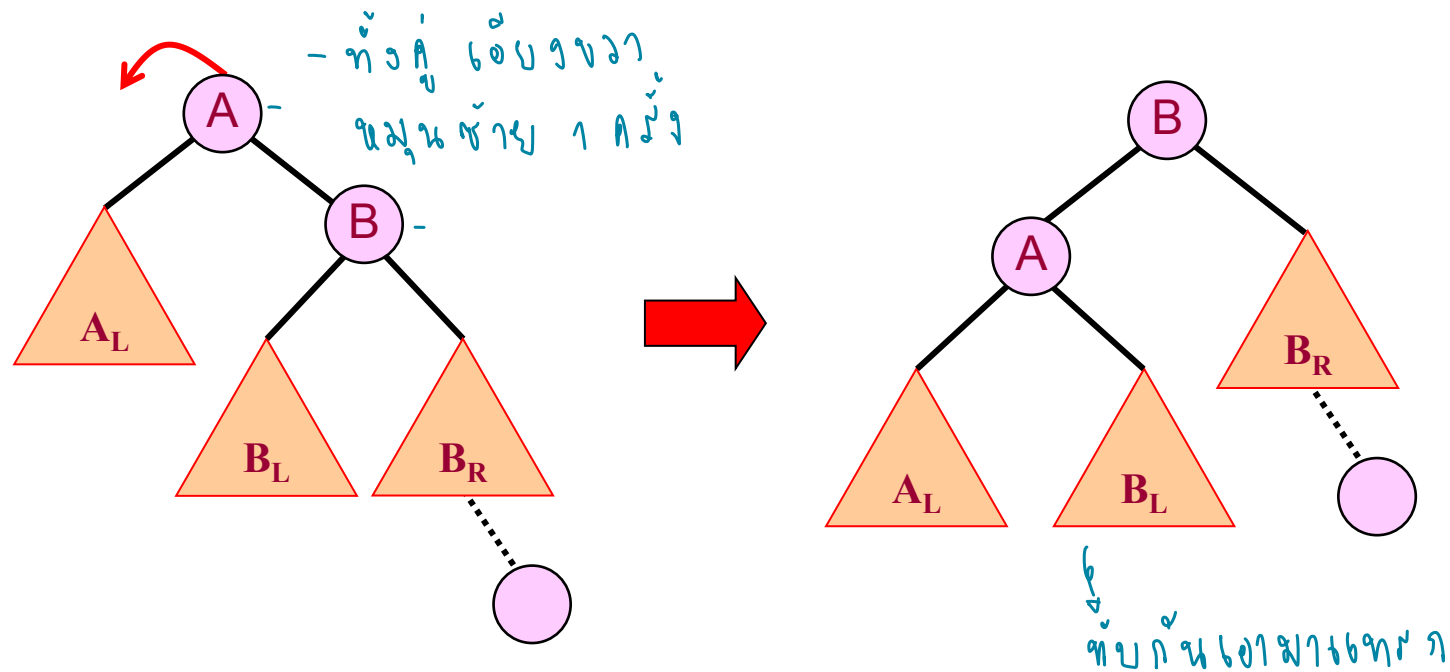
หมุน 1 ครั้ง $\rightarrow$ (+ เหนือกัน / - เหนือกัน)

↳ ถ้าไม่เอน
หมุน 2 ครั้ง

ย้ายมาขวา
ที่ฝั่งซ้ายของ B

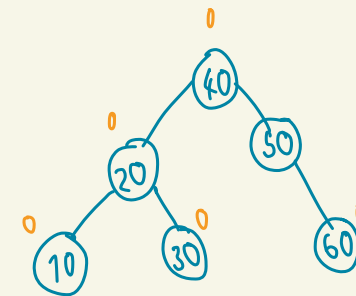
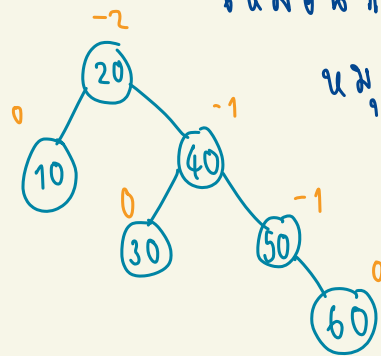
CASE 2 : RIGHT TO RIGHT

- หมุนซ้าย 1 ครั้ง (หมุนโหนด A)
 - โหนด A และ B เบ้ขวา (ค่าน้ำหนักเป็นลบทั้งคู่)



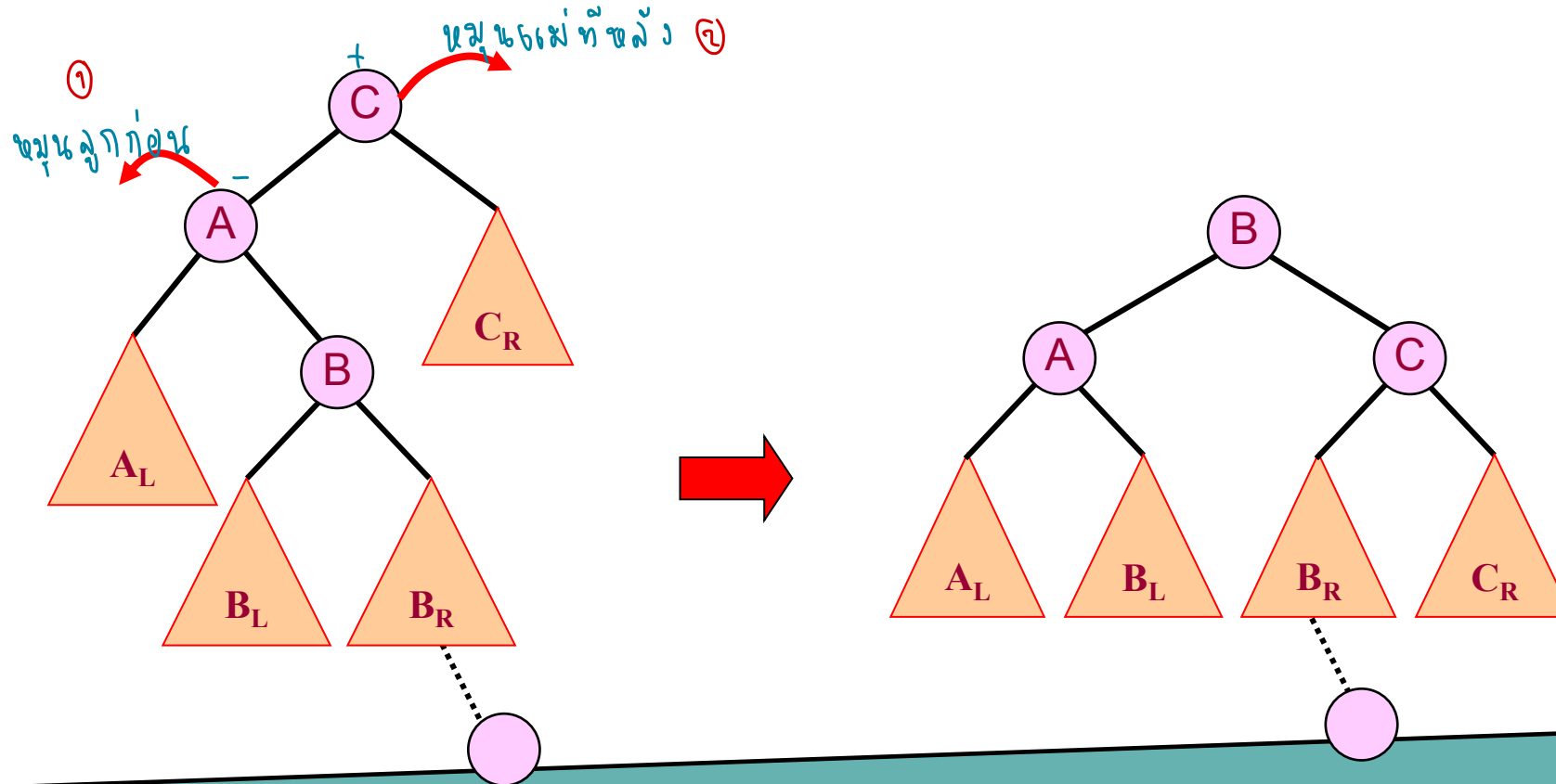
Case 2 :

- 6 นัสนั่น ๖๖๖๖๖๖๖๖
นัสนั่น ๖๖๖๖๖๖๖๖



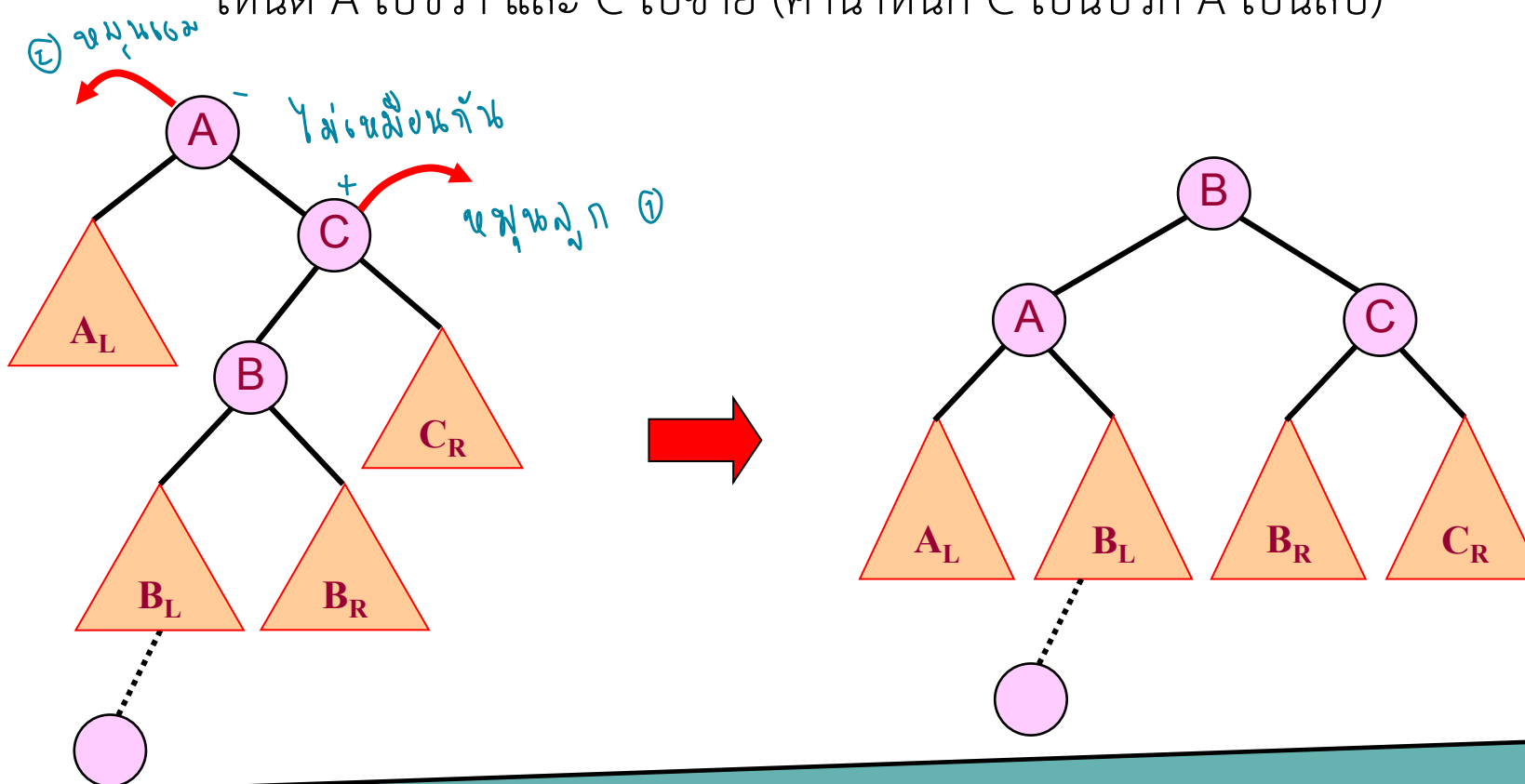
CASE 3 : RIGHT TO LEFT

- หมุนซ้าย 1 ครั้ง (หมุนโหนด A) หมุนขวา 1 ครั้ง (หมุนโหนด C)
 - โหนด C เบ้ซ้าย และ A เบ้ขวา (ค่าน้ำหนัก C เป็นบวก A เป็นลบ)

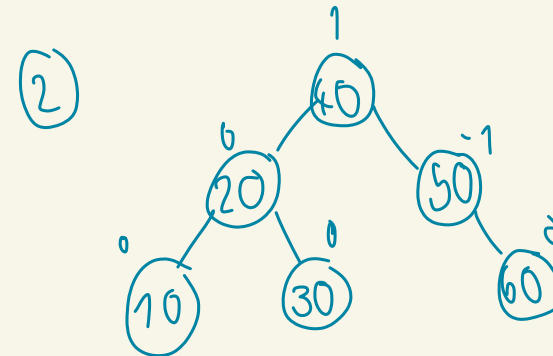
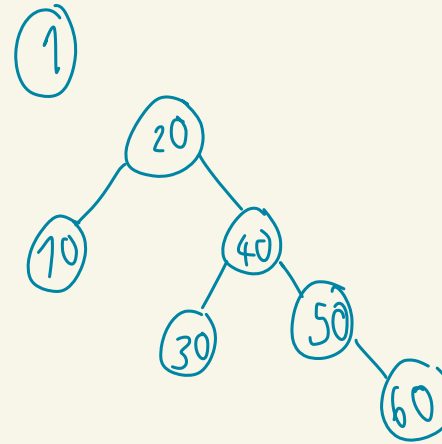
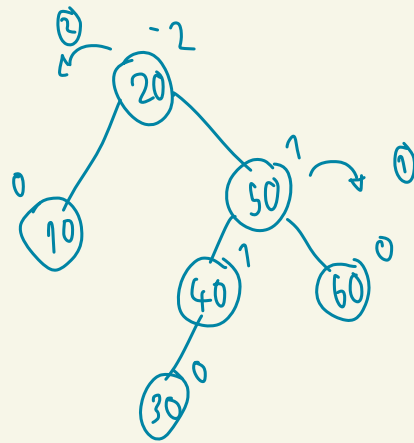


CASE 4 : LEFT TO RIGHT

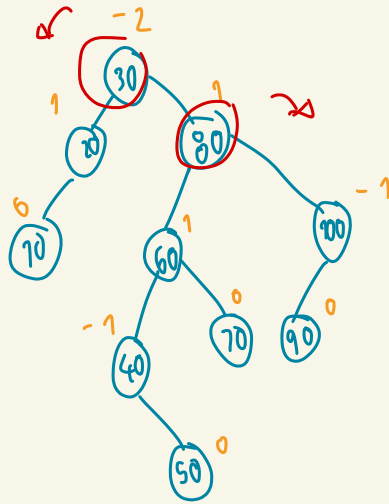
- หมุนขวา 1 ครั้ง (หมุนโหนด C) หมุนซ้าย 1 ครั้ง (หมุนโหนด A)
 - โหนด A เบ้ขวา และ C เบ้ซ้าย (ค่าน้ำหนัก C เป็นบวก A เป็นลบ)



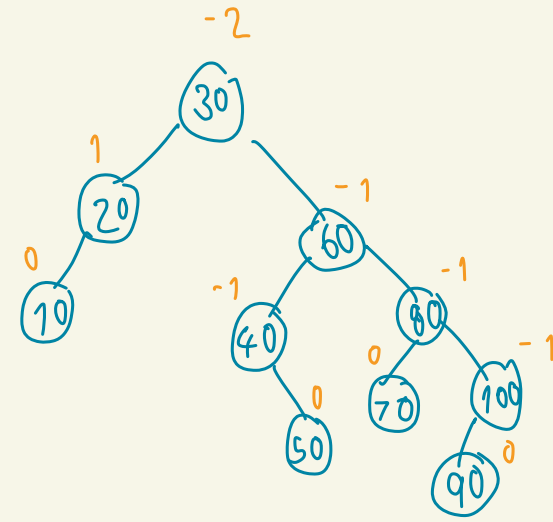
Case 4 :



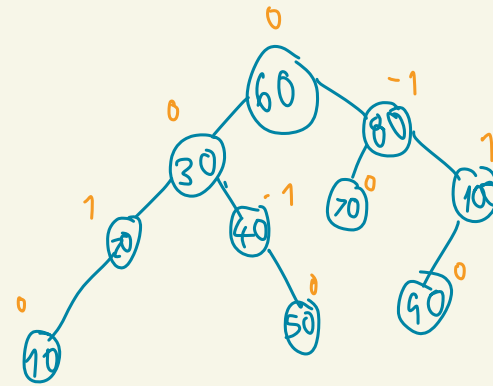
Exercise



①

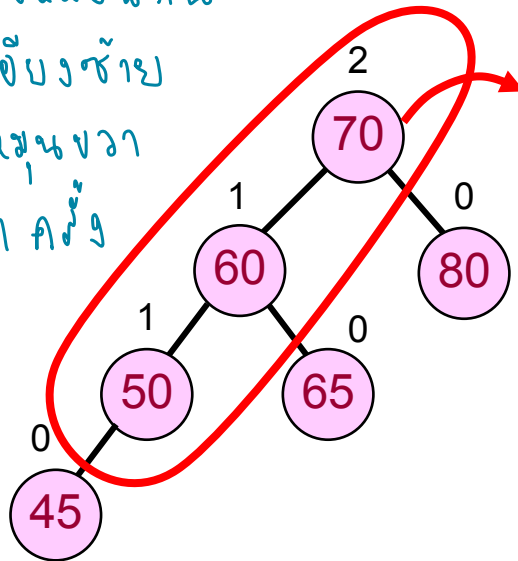


②

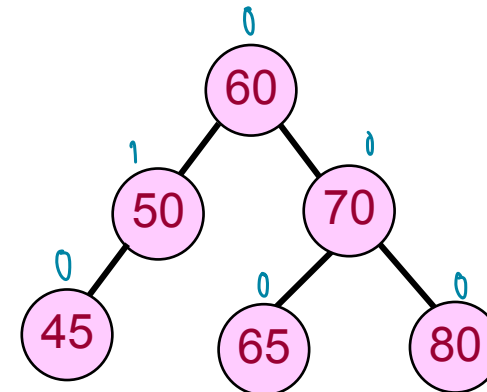
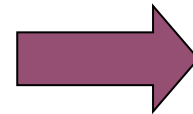


EXAMPLE : BALANCING TREE

+ 60 มีน้อง 2 คน
60 มีน้องซ้าย
50 มีน้องขวา
1 คน



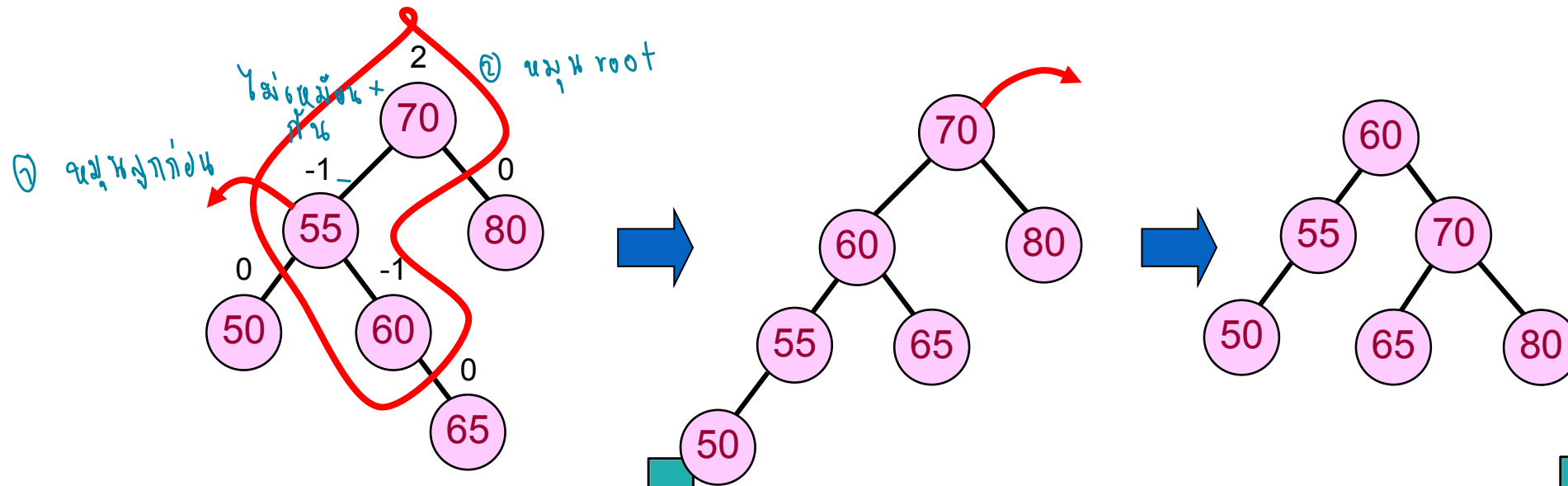
Case 1 : Left to Left



↓
60 มีน้อง 2 คน
50 มีน้องซ้าย
45 มีน้องขวา

EXAMPLE : BALANCING TREE

Case 3 : Right to Left



INSERT IN AVL TREE

- แทรกเหมือนที่ทำใน BST เพียงแต่เมื่อแทรกเสร็จแล้ว ถ้า BST ไม่สมดุล ก็ให้ปรับใหม่ตามกฎ

Algorithm AVLInsert (root, newData)

Using recursion, insert a node into an AVL tree.

Pre root is pointer to first node in AVL tree/subtree

 newData is pointer to new node to be inserted

Post new node has been inserted

Return root returned recursively up the tree

1 if (subtree empty)

 Insert at root

 1 insert newData at root

 2 return root

2 end if

3 if (newData < root)

 1 AVLInsert (left subtree, newData)

 2 if (left subtree taller)

 1 leftBalance (root)

 3 end if

4 else

 New data >= root data

 1 AVLInsert (right subtree, newPtr)

 2 if (right subtree taller)

 1 rightBalance (root)

 3 end if

5 end if

6 return root

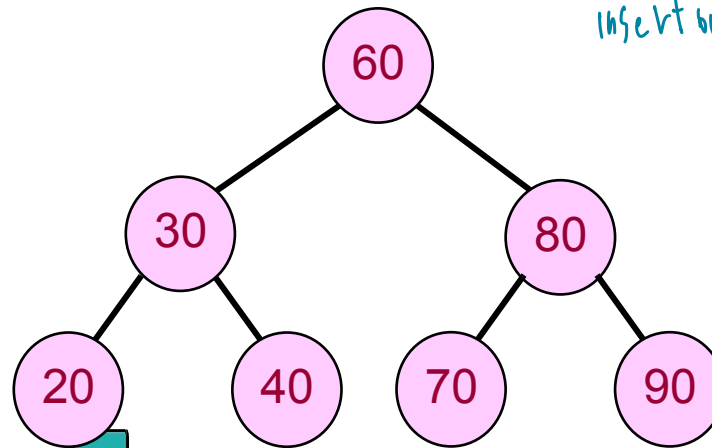
end AVLInsert

EXERCISE 1

Insert

- ให้เพิ่มค่า 25, 66 และ 97 เข้าไปใน AVL Tree ดังรูป แล้ววาดภาพผลลัพธ์ พร้อมแสดงค่าน้ำหนักความสูงในแต่ละโหนดด้วย

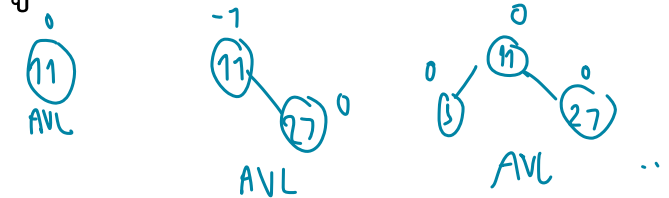
insert 66 กับ 97 เข้า Balance ทุกครั้ง



EXERCISE 2-3

insert

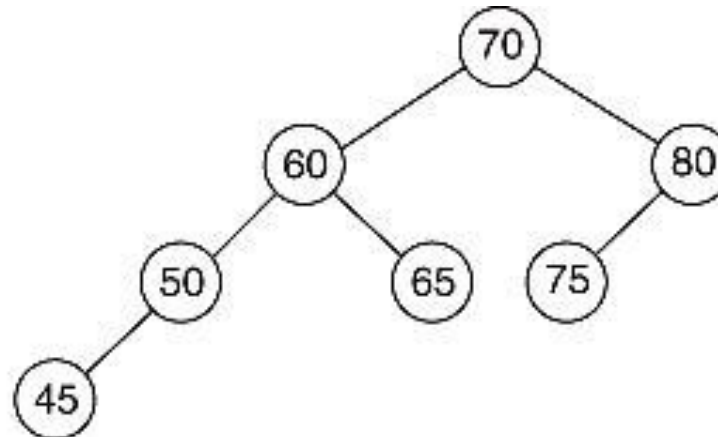
- จงสร้าง AVL Tree เมื่อได้รับข้อมูลตามลำดับต่อไปนี้ (วาดภาพการแทรกข้อมูลทีละตัว พร้อมแสดงค่าน้ำหนักความสูงในแต่ละโหนดด้วย)



Delete

11 27 5 9 44 59 88 70 75

- ให้ลบโหนดที่มีค่า 70 ออกจาก AVL Tree แล้ววาดภาพ AVL Tree ผลลัพธ์ พร้อมแสดงค่าน้ำหนักความสูงในแต่ละโหนดด้วย



Delete from BST